

Chapter 9

Machine Learning Algorithms

In this chapter of machine learning algorithms we will cover the basic algorithms of machine learning with their implementation using dataset and discuss their pro's and cons of algorithms for better understanding and how to use them in any project or with any dataset to get an correct result which can useful for research or study.

Key Component

We defined a data set composed of audio, images and binary labels that provided an understanding of how we could train a model from snippets to classifications. This sort of problem is one of many kinds of machine learning problems when we try and predict a specified unknown label given known inputs, provided that a dataset of examples for which labels are known is called supervised learning. Let's discuss what are the major key components we have to get there

- Data
- Models
- Algorithms

Linear Regression

Linear regression is a linear method in statistics to model the relationship between one or more independent variables and one (y) dependent variable (s). The relationships are modeled in linear regression with linear prediction functions which are estimated by unknown model parameters from the data. Linear regression is one of Machine Learning's most common algorithms. It is because of its relative simplicity and popular properties. It is two types which are

- **Simple Linear Regression:** If you only work with one independent variable, linear regression is called simple.

It is Denoted by the formula $f(x) = mx + b$

We will use two functions like cost and gradient to calculate and optimize our model with the data we provide.

Cost Function: We calculate the precision with the mean squared error (squared) cost function of our linear regression algorithm. The average squared distance between the expected and the real performance is calculated by MSE.

Its formula is

$$\text{Error}(m, b) = \frac{1}{N} \sum_{i=1}^N (\text{Actual output} - \text{Predicted output})^2$$

MSE is very simple and can be easily coded with Python like

```
def cost_function (m, b, x, y):
    totalError = 0
    for i in range (0, len (x)):
        totalError += (y[i]- (m*x[i]+b))**2
    return totalError/float (len (x))
```

Optimization: We will use gradient descent to find the coefficients which minimize our error function. Gradient descent is an algorithm of optimization that iteratively takes steps towards the local cost function.

In order to find the way to the bottom, we have to take the error function in relation to our m and b-slope. Then we take a step in the negative direction.

General Gradient Descent Formula

$$\Theta_j = \Theta_j - \alpha \frac{\partial}{\partial \Theta_j} J(\Theta_0 - \Theta_1)$$

Gradient Descent for Simple Linear Regression.

$$\frac{\partial}{\partial m} = \frac{2}{N} \sum_{i=1}^N -x_i (y_i - (mx_i + b))$$

$$\frac{\partial}{\partial b} = \frac{2}{N} \sum_{i=1}^N -(y_i - (mx_i + b))$$

The gradient descent implementation is a little more complex, but can also be easily accomplished in pure Python

```
def gradient_descent (b, m, x, y, learning_rate, num_iterations):
    N = float (len (x))
    for j in range (num_iterations): # repeat for num_iterations
        b_gradient = 0
        m_gradient = 0
        for i in range (0, len (x)):
            b_gradient += - (2/N) * (y[i] - (m * x[i]) + b))
            m_gradient += - (2/N) * x[i] * (y[i] - (m * x[i]) + b))
        b -= (learning_rate * b_gradient)
        m -= (learning_rate * m_gradient)
        if j%50==0:
            print ('error:', cost_function (m, b, x, y))
    return [b, m]
```

Implementing Linear Regression

We must construct a dataset and identify several initial variables in order to run our linear regression model. For simple random and matplotlib data to be shown, we use NumPy.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace (0, 100, 50)
delta = np.random.uniform (-10, 10, x.size)
y = 0.6*x + 5 + delta
plt.scatter (x, y)
plt.show ()
```

Output: as in Fig. 9.1

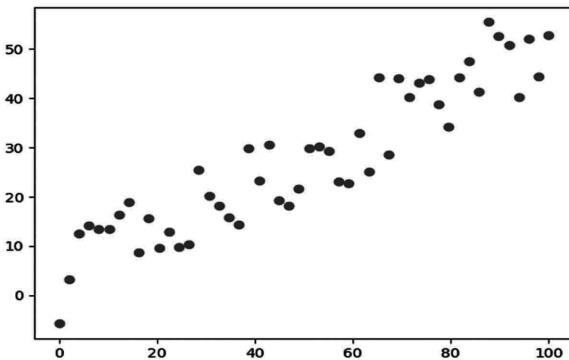


Fig. 9.1 Program Output

Now that we have our data, we are ready to use the above functionality to train our model:

```
# defining some variables
learning_rate = 0.0001
initial_b = 0
initial_m = 0
num_iterations= 100

print ('Initial error:', cost_function (initial_m,
                                       initial_b, x, y))

[b, m] = gradient_descent (initial_b, initial_m, x, y,
                           learning_rate, num_iterations)

print ('b:', b)
```

```
print ('m:', m)
print ('error:', cost_function (m, b, x, y))
plt.show ()
```

Output:

```
Initial error: 1576.099063465165
error: 205.2155773301355
error: 41.73021903795362
b: 0.03851474971994741
m: 0.6744456473643745
error: 41.69053444198719
```

We can now use our model as a predictor, as we have trained it. We will only predict the training data in this simple example and use the findings to plot the best matplotlib line.

```
predictions = [ (m * x[i]) + b for i in range (len (x))]
plt.scatter (x, y)
plt.plot (x, predictions, color='r')
plt.show ()
```

Output: as in Fig. 9.2

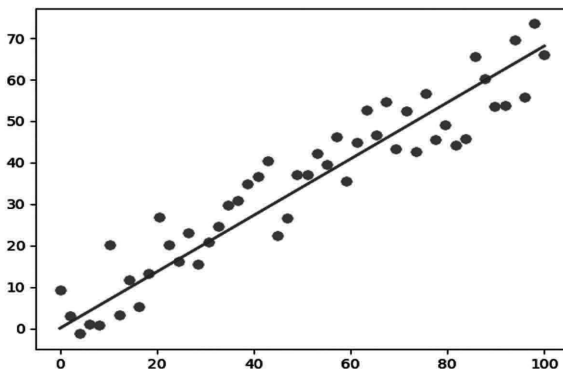


Fig. 9.2 Program Output

Multiple Linear Regression: When you deal with at least two independently operated variables, the linear regression is called multivariate. Each of the variables also known as characteristics is multiplied by a weight that our linear regression algorithm has learned.

It is Denoted by the formula

$$f(x) = b + W_1X_1 + W_2X_2 + \cdots W_nX_n = b + \sum_{i=0}^n W_iX_i$$

We will use two functions like cost and gradient to calculate and optimize our model with the data we provide.

Cost Function: We will use a mean squared error as a loss function, just as we did with a Simple linear regression. The only difference is that our prediction performance now comes from another feature.

As it is now possible to use NumPy to code our cost function with multiple features and weights:

Logistic Regression:

This is an algorithm of classification, used in categories of the response vary. The idea of the regression is to create a link between characteristics and the likelihood of a specific result.

For instance. The answer variable has two values, pass and fails when we need to predict if a student passes the test or fails if the number of hours studied is given as the factor.

This type of problem is called a binomial logistical regression, in which the answer variable has two 0 and 1 values or passes and fails or is true and false. The Multinomial Logistic Regression discusses situations where three or more potential values can be present in the response variable.

Why Logistic Regression, But not Linear?

Let 'x' be some function with binary classification, and let 'y' be the output that can be either 0 or 1.

In view of the input, the probability that the output is 1 is:

$$P(y = 1/x)$$

We may state that if we estimate the probability by means of linear regression:

$$p(X) = \beta_0 + \beta_1 X$$

where,

$$p(x) = p(y = 1/x)$$

The predicted probability can be produced by a linear regression model as any number ranges from negative to positive, while an outcome may be only between $0 < P(x) < 1$ as in Fig. 9.3.

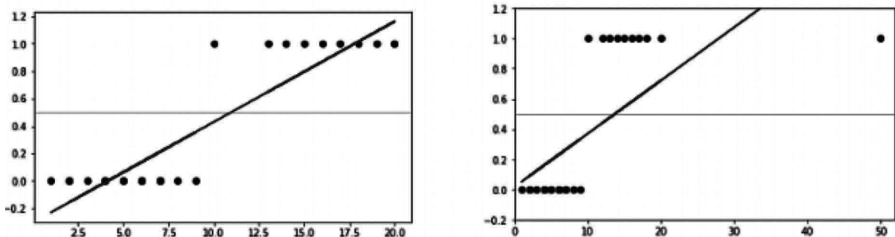


Fig. 9.3 Logistic Regression

Linear regression also has a huge effect on outliers.

To avoid this problem, log-odds function or logistic function is used

Logistic/logit Function

Logistic regression can be Denoted as:

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X$$

The logit or log-odds function is on the left side, and $p(x)/(1-p(x))$ is referred to as odd.

The chances represent the ratio of success probability to loss probability. Therefore, a linear combination of inputs with the log (odds) in the Logistic Regression is mapped-output equal to 1.

If the above function is inversed then we get:

$$p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$

This equation is known to be Sigmoid function which is a curve that provides an S-shaped curve. It also provides a probability value of $0 < p < 1$.

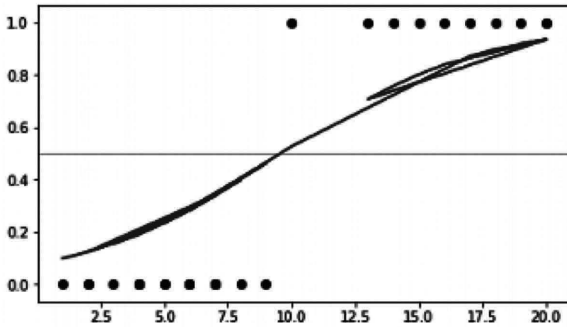


Fig. 9.4 Sigmoid Function

Estimation of Regression Coefficients

We use Maximum Likelihood Estimation, unlike linear regression, which uses Ordinary Last Square for parameter estimation.

There can be limitless collections of coefficients of regression. The maximums probability is that the maximum possibility of obtaining the data we have observed is a series of regression coefficients.

If we have binary results, it's only β if the result was good and $1-\alpha$ otherwise. We have the probability function therefore:

$$\mathcal{L}(\beta; y) = \prod_{i=1}^N \left(\frac{\pi_i}{1-\pi_i} \right)^{y_i} (1-\pi_i)$$

To evaluate the parameter value, the probability function log is adopted because it does not modify the function properties.

The log-like nature is distinguished, and parameter values that maximize the log-likelihood are calculated using iterative techniques such as Newton.

Performance of Logistic Regression Model: Deviance is used instead of sum of squares to test the output of the logistic regression model as seen in table 9.1.

- Null Deviance demonstrates the expected reaction of a model with an interception only.
- System deviation indicates the answer to the addition of independent variables expected by a system. The parameter or set of parameters can be inferred that the model’s variance is substantially smaller than the null deviance.
- The Confusion Matrix is another way to assess the accuracy of the model.

Table 9.1 Logistic Regression model Performance

	P' (Predicted)	n' (Predicted)
P (Actual)	True Positive	False Negative
n (Actual)	False Positive	True Negative

The Accuracy of the model is defined by:

$$\frac{\text{True Positives} + \text{True Negatives}}{\text{True Positive} + \text{True Negatives} + \text{Flase Positives} + \text{Flase Negatives}}$$

Multi-class Logistic Regression

Behind multi-class and binary regression, there is the same underlying intuition. However, we pursue a one v/s approach to multi-class issues.

For instance. We deal with a Multi-Class Problem if we have to predict if the weather is sunny, rainy or windy. We translate this into three question of binary classification, namely snowy and snowy, rainy or not, and windy or not. All three classifications are performed on feedback independently. The solution is the form for which the probability value is greater compared to others.

Implementation of Logistic Regression

We will simply implement logistic regression using Scikit-learn library which will allow us to use predefined logistic regression model. We will use iris Dataset:

The Iris dataset is perhaps one of the simplest machine learning datasets. The Iris dataset describes iris flowers in numerical form. It records sepal and petal length/width measurements can be seen in Fig. 9.5. Using these steps, we will try to predict flower species.

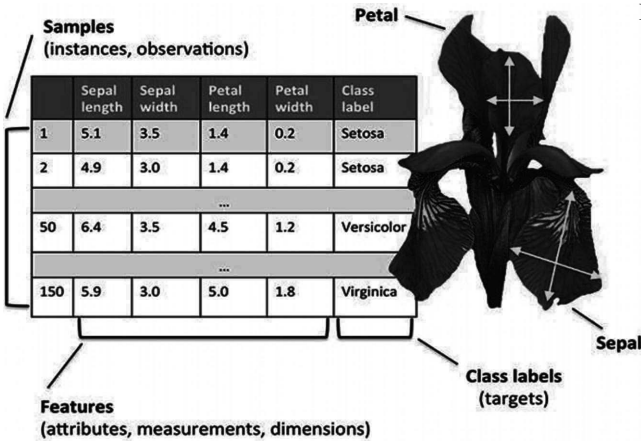


Fig. 9.5 Dataset Description

Which has columns regarding sepal length cm, sepal width cm, petal length cm, petal width cm, species in which first four Column are labels and last one is the target.

You can run these code in any python environment or Editor.

Step-1: we will import all the libraries required for implementation

```
# import dependencies
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Note: The symbol # represent the comment for the program which meant for explanations of code

Step-2: loading the data

```
data = datasets.load_iris ()
```

here as we have imported the data through sckit's package, we will get the data in form of arrays,

```
# so here our labes is stored in
data.data
```

Output:

```
array ([[5.1, 3.5, 1.4, 0.2], [4.9, 3., 1.4, 0.2], [4.7, 3.2, 1.3,
0.2],[4.6, 3.1, 1.5, 0.2], [5., 3.6, 1.4, 0.2], [5.4, 3.9, 1.7, 0.4],[4.6,
3.4, 1.4, 0.3],[5., 3.4, 1.5, 0.2],[4.4, 2.9, 1.4, 0.2], [4.9, 3.1, 1.5,
0.1], [5.4, 3.7, 1.5, 0.2], [4.8, 3.4, 1.6, 0.2], [4.8, 3., 1.4, 0.1],
[4.3, 3., 1.1, 0.1], [5.8, 4., 1.2, 0.2], [5.7, 4.4, 1.5, 0.4], [5.4, 3.9,
1.3, 0.4], [5.1, 3.5, 1.4, 0.3], [5.7, 3.8, 1.7, 0.3],.....so on in this
way it will print all the data in the form of array on the screen.
```

```
# Targets for labels
```

```
data.target
```


Output:

```
array ([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0,0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,1,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

Step 3: Now we will divide the data

we will divide the dataset into data and target for the model

```
X = data.data[:, :2] # X is the features in our dataset
```

```
y = (data.target != 0) * 1 # y is the Labels in our dataset
```

After separating we will visualize it in a graph using matplotlib

here we have only taken the x line as SepalLengthCm and y line as SepalWidthCm

```
plt.figure (figsize= (10, 6))
```

```
plt.scatter (X[y == 0][:, 0], X[y == 0][:, 1], color='g', label='0')
```

```
plt.scatter (X[y == 1][:, 0], X[y == 1][:, 1], color='r', label='1')
```

```
plt.legend ();
```

Output: As in Fig. 9.6

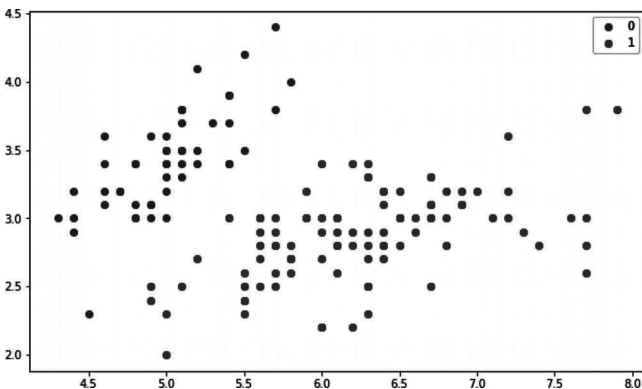


Fig. 9.6 Program Output

Step 4: Now will using scikit-learn logistic regression model to train and predict.

```
# import library with model
```

```
from sklearn.linear_model import LogisticRegression
```

```
# loading the model which c as parameter which can be given to
strengthen the model with predefined values like learning rate,
iteration etc.,
```

```
model = LogisticRegression (C=1e20)
# now we will train the model
model.fit (X, y)
```

Output:

```
LogisticRegression (C=1e+20, class_weight=None, dual=False,
                    fit_intercept=True,
                    intercept_scaling=1, l1_ratio=None, max_iter=100,
                    multi_class='auto', n_jobs=None, penalty='l2',
                    random_state=None, solver='lbfgs', tol=0.0001, verbose=0,
                    warm_start=False)

# Prediction from model
preds = model.predict (X)
(preds == y).mean ()
```

Output:

1.0

Pros:

- Simple and efficient.
- Low variance.
- It provides probability score for observations.
- It can also be used for extraction of features

Cons:

- Does not handle several categorical characteristics/ attributes properly.
- It needs to transform non-linear characteristics.
- They are not sufficiently robust to handle more complex connections.

Decision Tree

The algorithm of the decision tree comes under the supervised learning category. A decision tree can be used to represent decisions and decision-making visually and clearly. This uses a tree-like decision pattern as the name goes. Although a commonly used data mining technique for a strategy to achieve a certain goal, it is often used extensively for machine learning.

Decision trees are designed using algorithms that recognize ways to break a collection of data based on various criteria. Classification trees are called tree models in which the target variable will take a distinct set of values. Decision trees where continuous values (typically genuine numbers) can be taken from the target variable are called regression trees. The general term for this is Classification and Regression Tree (CART).

Important Terminology in Decision Tree

Let us discuss the fundamental terms used in decision trees which can be seen in Fig. 9.7.

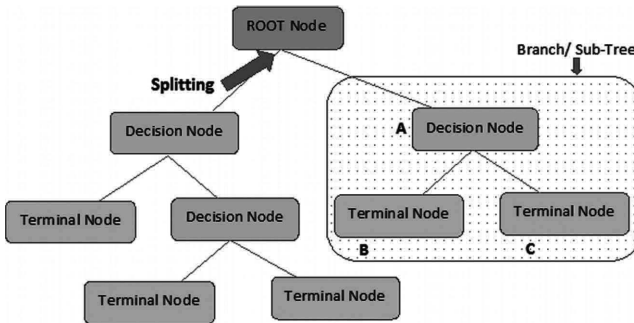


Fig. 9.7 Decision tree representation

Note:- A is parent node of B and C.

- **Root node:** It represents a whole population or sample and is further split up into two or more uniform samples.
- **Splitting:** This is a mechanism by which a node is split into two or more sub nodes.
- **Decision Node:** If a sub node splits into additional sub nodes, it is referred to as a decision node.
- **Terminal Node/Leaf:** Nodes without children (no further division) are referred to as Leaf nodes and Terminal nodes.
- **Pruning:** when we reduce the size of decision trees by eliminating nodes (the opposite to splitting).
- **Branch/Sub-Tree:** It is referred to as the sub-section of decision-tree.
- **Parent and Child Node:** A node divided into sub-nodes is called the parent node of sub-nodes where the child of a parent node is the sub-node.

Decision Tree Making Approach

The biggest challenge in Decision Tree is to classify the attribute for each level of the root node. This method is referred to as selection of attributes. We have two common selection measures for attributes:

- **Information Gain:** When we use a node in a decision node, the training instances are split into smaller entropy sub-sets. This shift in entropy is determined by the gain of information. We ask various types of questions in every node of the tree during the decision process. On the basis of the question asked, we will measure the data gain.

Definition: Suppose S is a set of instances, A is an attribute, S_v is the subset of S with $A = v$, and $\text{Values}(A)$ is the set of all possible values of A , the

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

Entropy: Entropy is the uncertainty factor for a random variable and it describe the impurity of arbitrary instances. The greater the entropy, the greater the value of knowledge.

- **Gini Index:** The Gini index is a measure of how frequently a randomly selected variable will be defined incorrectly. This means that you would choose an attribute with a lower Gini index. Sklearn follows “Gini” index parameters and takes “gini” value by chance. The Formula is given below to calculate the Gini Index.

$$\text{Gini Index} = 1 - \sum_j p_j^2$$

Implementation of Decision Tree

Let’s take a look at how a decision tree classification in Python can be implemented. To continue,

Step-1: we import the libraries.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
from sklearn.tree import export_graphviz
from sklearn.externals.six import StringIO
from IPython.display import Image
from pydot import graph_from_dot_data
import pandas as pd
import numpy as np
```

Step-2: Loading the dataset

```
iris = load_iris ()
X = pd.DataFrame (iris.data, columns=iris.feature_names)
y = pd.Categorical.from_codes (iris.target, iris.target_names)
X.head ()
```

Output:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Step-3: Splitting the data

```
y = pd.get_dummies (y)
X_train, X_test, y_train, y_test = train_test_split (
    X, y, random_state=1)
```

Step-4: Using the decision tree classifier to classify the flowers

```
dt = DecisionTreeClassifier ()
dt.fit (X_train, y_train)
```

Output:

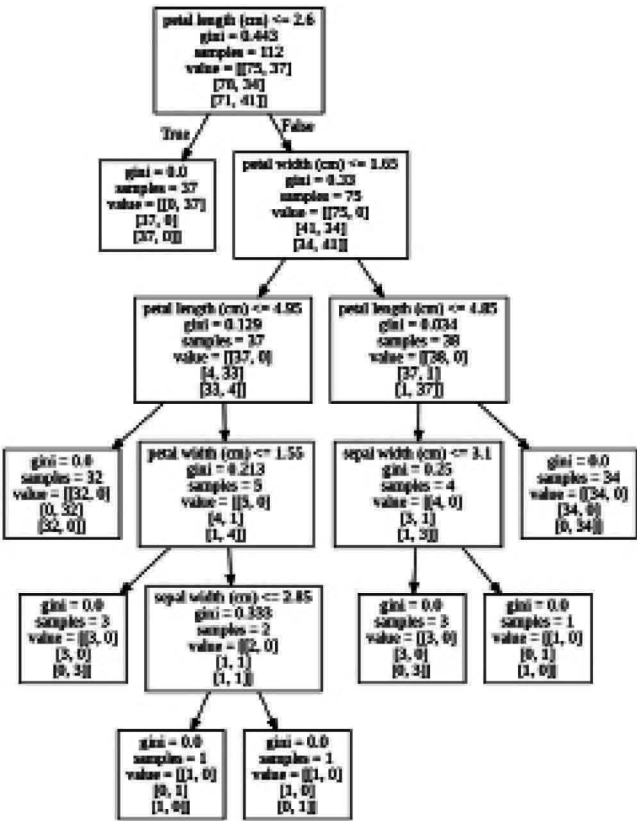
```
DecisionTreeClassifier (ccp_alpha=0.0, class_weight=None,
                        criterion='gini',
                        max_depth=None, max_features=None, max_leaf_nodes=None,
                        min_impurity_decrease=0.0, min_impurity_split=None,
                        min_samples_leaf=1, min_samples_split=2,
                        min_weight_fraction_leaf=0.0, presort='deprecated',
                        random_state=None, splitter='best')
```

Step-5: Compiling and visualizing the decision tree

```
dot_data = StringIO ()
export_graphviz (dt, out_file=dot_data,
                 feature_names=iris.feature_names)
(graph,) = graph_from_dot_data (dot_data.getvalue ())
Image (graph.create_png ())
```

Note how it includes Gini impurities, the total sample number, the criteria for classification and the number of left/right samples.

Output:



Step-6: Prediction with confusion matrix

```

y_pred = dt.predict (X_test)
species = np.array (y_test).argmax (axis=1)
predictions = np.array (y_pred).argmax (axis=1)
confusion_matrix (species, predictions)

```

Output:

```

array([[13, 0, 0],
       [ 0, 15, 1],
       [ 0, 0, 9]])

```

We can see that our decision tree listed 37/38 plants correctly.

Support Vector Machine

Support Vector Machine (SVM) is a categorized and regression analysis machine learning approach. It relies on supervised models of learning and training by algorithms. The vast volumes of data are analyzed for trends.

An SVM produces two parallel lines for parallel partitions. For each data category, it utilizes almost every attribute in a high-dimensional field. It separates the space for flat and linear partitions in a single pass. Divide the 2 groups into the clearest possible distance. Do so by a plane known as a hyperplane.

An SVM generates hyperplane with the highest margin to divide data into classes in a high-dimensional area. The distance between the two classes is the highest distance between such classes' nearest data points.

The bigger the margin, the lower the classifier classification generalization error is as in Fig. 9.8:

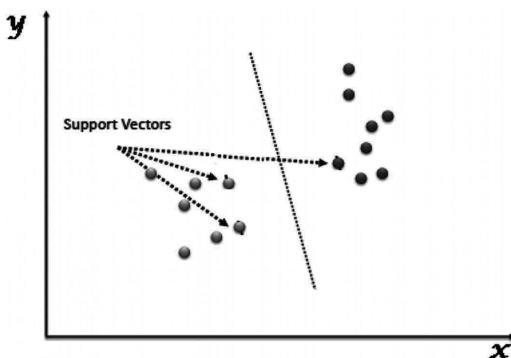


Fig. 9.8 SVM Representation

How does it work?

Consider two conditions in order to grasp the SVM algorithm:

- Separate case – Infinite boundaries can be split into two groups.
- Non-separable case – There is no division between two groups, but one overlap.

Implementation of SVM

Step 1: Importing the libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report,
    confusion_matrix
```

Step 2: importing and Using the dataset from google drive

```
from google.colab import drive
drive.mount ('/gdrive')
%cd /gdrive
```

Output:

Go to this URL in a browser: https://accounts.google.com/o/oauth2/auth?client_id=94.....y

Enter your authorization code:

.....

Mounted at /gdrive

This is the code for mounting your drive with google research Collaboratory which will ask you authentication which you can do it with your Gmail account by just following the simple steps. As you can see in the output how Collaboratory will ask you to verify it by clicking in the given link then it will get mounted by giving a authentication to it.

```
#if you are using python editor just provide the file path which
is where you have stored on your system
# Import the dataset using Seaborn library
iris=pd.read_csv ('/gdrive/My Drive/Colab Notebooks/IRIS.csv')
#providing path for dataset

# Checking the dataset
iris.head ()
```

output:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Step 3:Visualizing the similarities into the dataset using pair plots

```
# Creating a pairplot to visualize the similarities and especially
difference between the species
sns.pairplot (data=iris, hue='species', palette='Set2')
```

Output: as in Fig. 9.9

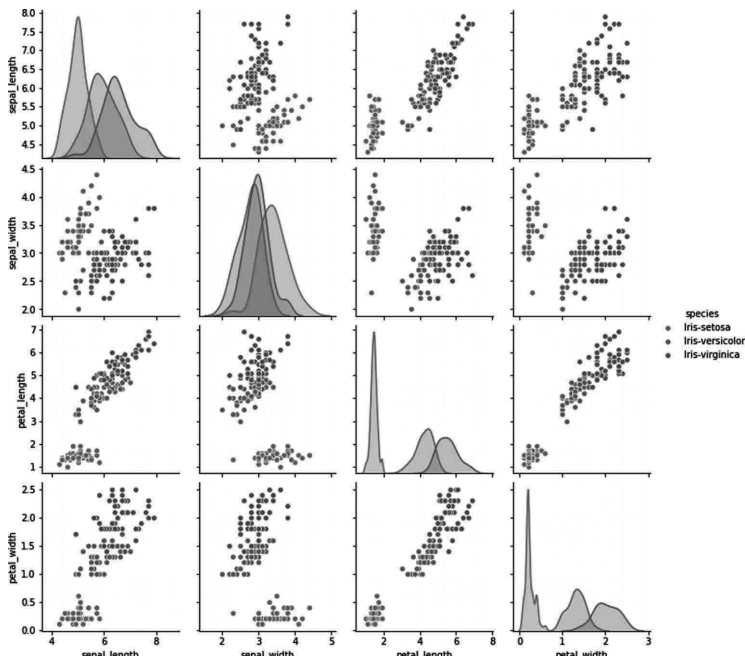


Fig. 9.9 Program output

Step 4:Splitting the test and train data

```
# Separating the independent variables from dependent variables
x=iris.iloc[:, :-1]
y=iris.iloc[:, 4]
x_train,x_test, y_train, y_test=train_test_split
(x,y,test_size=0.30)
```


Step 5: Loading model and training

```
model=SVC ()
model.fit (x _train, y _train)
```

Output:

```
SVC (C=1.0, break _ties=False, cache _size=200,
     class _weight=None, coef0=0.0, decision _function _
     shape='ovr', degree=3, gamma='scale', kernel='rbf',
     max _iter=-1, probability=False, random _state=None,
     shrinking=True,tol=0.001, verbose=False)
```

Step 6: Results of prediction

```
pred=model.predict (x _test)
# Importing the classification report and confusion matrix
print (confusion _matrix (y _test,pred))
```

Output:

```
[[15 0 0]
 [ 0 14 0]
 [ 0 2 14]]
#generating Classification report
```

```
print (classification _report (y _test, pred))
```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	15
Iris-versicolor	0.88	1.00	0.93	14
Iris-virginica	1.00	0.88	0.93	16
accuracy			0.96	45
macro avg	0.96	0.96	0.96	45
weighted avg	0.96	0.96	0.96	45

Naïve Bayes

Naive Bayes algorithm can be described as a supervised Classification algorithm based on Bayes theorem with independence between features. It is also a technique of classification.

Bayes theorem is a principle for constructing classifiers behind this classification technique. The predictors are expected to be independent. In simple terms, the inclusion in a class of a particular characteristic is not connected to that of any other characteristic. The following is the Bayes theorem equation:

$$P(A|B) = P(B|A) P(A)/P(B)$$

Where

- $P(A|B)$ is the probability of hypothesis A given the data B. This is called the posterior probability.
- $P(B|A)$ is the probability of data B given that the hypothesis A was true.

- $P(A)$ is the probability of hypothesis A being true (regardless of the data). This is called the prior probability of A.
- $P(B)$ is the probability of the data (regardless of the hypothesis).

Implementation

Python libraries provide three types of classifiers of Naïve Bayes:

- **Gaussian:** It assumes that the characteristics are distributed accordingly.
- **Multinomial:** For discrete counts,
- **Bernoulli:** If the function vectors are binary, the binomial model is useful (e.g. zeros and vectors)

In implementation we will be Gaussian naïve Bayes classifier and iris dataset. We already know the iris dataset as we have studied about it in before algorithms.

Of course, in this example only one and two classes are listed. However, also with a vector with many features and groups the process is the same (the only exception being the use of an Argmax function to return the most probable).

Step 1: Importing all libraries

```
from sklearn import datasets
import matplotlib.pyplot as plt
import pandas as pd
#importing the necessary packages
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
```

Step 2: Dataset loading and Splitting

```
#downloading the iris dataset, splitting it into train set and
validation set
iris = datasets.load_iris ()
class_names = iris.target_names
iris_df=pd.DataFrame (iris.data, columns=iris.feature_names)
iris_df['target']=iris.target
iris_df.head ()
```

Output:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
iris_df.describe ()
```

Output:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

We have 4 features and a categorical goal because we have 3 levels (Setosa, Versicolor and Virginica, 0, 1 and 2). We can now build our classifiers accordingly:

Now Splitting the data into test and train

```
X_train, X_test, y_train, y_test = train_test_split (iris_
df[['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
'petal width (cm)']], iris_df['target'], random_state=0)
```

Step 3: Loading Model and compilation

```
NB = GaussianNB ()
NB.fit (X_train, y_train)
y_predict = NB.predict (X_test)
print ("Accuracy Naive Bayes: {:.2f}".format (NB.score (X_test,
y_test)))
```

Output:

Accuracy Naive Bayes: 1.00

As you can see, all test data are categorized correctly by our algorithm. It's a nice performance, the best one we will ever achieve. But take into account our weak dataset dimension (only 150 observations). Standard precision of 100 percent on real data is difficult to achieve. In addition, the argument for an error equal to zero may also be counterproductive. Indeed we prefer to have an algorithm that can generalize and not conform perfectly to test data, because the last goal of any ML algorithm is to make accurate predictions of new unknown data. The risk of overruns will lead to an unsustainable model if new data cannot be used (using too many parameters to create a model that perfectly fits test data).

Pros:

- Easy to comprehend.
- It can be trained on small data sets as well.

Cons:

- There are many sample corrections to solve this problem, such as “Laplacian correction,” for features that have a null-frequency, the total likelihood is zero.
- Another drawback is that it very strongly assumes independence class features. These data sets in real life are almost difficult to locate.

K-Nearest Neighbors (KNN)

It is used to identify problems and to rectify them. It is used commonly to solve problems with classification. The key principle of this algorithm is that it stores all the existing cases and by voting plurality of its neighbors classifies new cases. The case is then assigned to the class which, according to a distance function, is the most common among his nearest K neighbors. The gap can be from Minkowski, Manhattan, Euclidean, and Hamming. Consider KNN as follows:

- Computationally KNN are expensive than other algorithms used for classification problems.
- The normalization of variables needed otherwise higher range variables can bias it.
- At KNN, we will work on a stage such as noise reduction, pre-processing.

Implementation

Step 1: Importing Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
```

Step 2: Loading And visualizing the data

```
names = ['sepal_length', 'sepal_width', 'petal_length',
         'petal_width', 'class']
# Link for downloading dataset: https://archive.ics.uci.edu/ml/
machine-learning-databases/iris/
df = pd.read_csv ('C:/Users/EETRO/Desktop/Book Code/iris.data.
txt', header = None, names = names) #path where file is stored
at desktop
df.head ()
```

Output:

	sepal_length	sepal_width	petal_length	petal_width	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
setosa = df[df['class'] == 'Iris-setosa']
versicolor = df[df['class'] == 'Iris-versicolor']
virginica = df[df['class'] == 'Iris-virginica']
```

```
plt.plot (setosa['sepal_length'], setosa['sepal_width'], 'ro',
label = 'setosa')
```

```
plt.plot (versicolor['sepal_length'], versicolor['sepal_width'],
'bo', label = 'versicolor')
plt.plot (virginica['sepal_length'],  virginica['sepal_width'],
'go', label = 'virginica')
plt.xlabel ('sepal_length')
plt.ylabel ('sepal_width')
plt.legend ()
plt.show ()
```

Output: as in Fig. 9.10

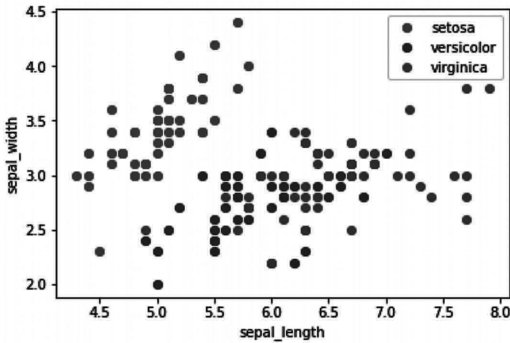


Fig. 9.10 Program Outcome

```
plt.plot (setosa['petal_length'], setosa['petal_width'], 'ro',
label = 'setosa')
plt.plot (versicolor['petal_length'], versicolor['petal_width'],
'bo', label = 'versicolor')
plt.plot (virginica['petal_length'],  virginica['petal_width'],
'go', label = 'virginica')
plt.xlabel ('petal_length')
plt.ylabel ('petal_width')
plt.legend ()
plt.show ()
```

Output: as in Fig. 9.11

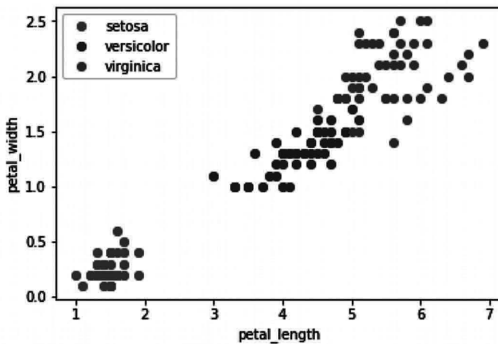


Fig. 9.11 Program outcome

```
len (df)
```

Output:

```
150
print ('length of setosa: %d' % (len (setosa)))
print ('length of versicolor: %d' % (len (versicolor)))
print ('length of virginica: %d' % (len (virginica)))
```

Output:

```
length of setosa: 50
length of versicolor: 50
length of virginica: 50
```

Step 3: Dividing the data into training and testing set with 3:1 ratio

```
df['is_train'] = np.random.uniform (0, 1, len (df)) <=.75
df.head ()
```

Output:

	sepal_length	sepal_width	petal_length	petal_width	class	is_train
0	5.1	3.5	1.4	0.2	Iris-setosa	True
1	4.9	3.0	1.4	0.2	Iris-versicol	True
2	4.7	3.2	1.3	0.2	Iris-versicol	False
3	4.8	3.1	1.5	0.2	Iris-versicol	True
4	5.0	3.6	1.4	0.2	Iris-versicol	True

```
train, test = df[df['is_train'] == True],
              df[df['is_train'] == False]
print ('length of train dataset: %d' % (len (train)))
print ('length of test dataset: %d' % (len (test)))
```

Output:

```
length of train dataset: 111
length of test dataset: 39
```

```
train.columns
```

Output:

```
Index (['sepal_length', 'sepal_width', 'petal_length',
'petal_width', 'class', 'is_train'], dtype='object')
```

```
train_x = train[train.columns[:len (train.columns) - 2]]
train_y = train['class']
test_x = test[test.columns[:len (test.columns) - 2]]
test_y = test['class']
```

Step 4: Calculation of distances by using function method
i.e., Euclidean Distance and Manhattan Distance

$$\text{Euclidean Distance} = \sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

$$\text{Manhattan Distance} = \sum_{i=1}^k |x_i - y_i|$$

```
#Euclidean Distance function takes inputs as  
#data1 : first data point  
#data2 : second data point
```

```
#and gives output as Euclidean distance
```

```
def euclidean_distance (data1, data2):  
    distance = 0  
    for i in range (data2.shape[0]):  
        distance += np.square (data1[i] - data2[i])  
    return np.sqrt (distance)
```

```
#Manhattan Distance function takes inputs as  
#data1 : first data point  
#data2 : second data point
```

```
#and gives output as Manhattan distance
```

```
def manhattan_distance (data1, data2):  
    distance = 0  
    for i in range (data2.shape[0]):  
        distance += abs (data2[i] - data1[i])  
  
    return distance
```

Step 5: Forming KNN function

Now KNN function takes inputs as

train_x : training samples,

train_y : corresponding labels,

dis_func : a function which calculates distance

sample : one test sample

k : number of training example to look for deciding the class of the given sample.

```
def knn (train_x, train_y, dis_func, sample, k):  
    distances = {}  
    for i in range (len (train_x)):
```



```
if predicted_label_manhattan == true_label:
    correct += 1
accuracy_manhattan = (correct/len (test_x)) * 100
print ("model accuracy with euclidean is %0.2f"
      % (accuracy_euclidean))
print ("model accuracy with manhattan is %0.2f"
      % (accuracy_manhattan))
get_accuracy (test_x, test_y, train_x, train_y, 5)
```

Output:

Model accuracy with euclidean is 94.87

model accuracy with manhattan is 94.87

Accuracy is the ratio of the number of instances correctly identified to the number of instances. Here, we can see that the Euclidean and Manhattan range as a measure of similarity is 94.87 percent accurate.

Pros:

- No data assumptions.
- Simple to understand algorithm.
- Can be used for regression and classification.

Cons:

- High memory demand — In order to measure nearest K neighbors, all training data should be in the memory.
- Related features sensitive.
- Sensitive to the size of data as the distance to the nearest K points is computed.

K-means Clustering

It is used to resolve the problem of clusters, as the name implies. This is a kind of unsupervised Learning. The principal principle of the K-Means algorithm is to assign the data set across different clusters. The K-means cluster algorithm is used to identify and find patterns and to make better decisions for groups that are not clearly labeled in the data. The new data can be easily allocated to the most appropriate group once an algorithm is performed and groups identified.

In order to start with k-means, you must initialize the cluster centroids (K) by random means. K-means is an iterative, two-step algorithm as in Fig. 9.12:

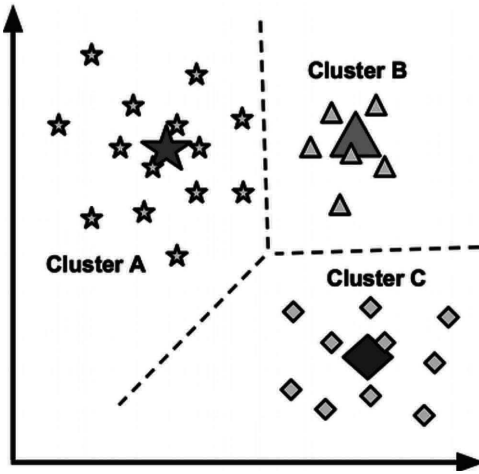


Fig. 9.12 Data Points Clustering

1. Assignment of the cluster: The algorithm traverses every data point and assigns data points to one of the 3 cluster centroids, depending on the cluster that is closer.
2. Move to centroid: Centroids are transferred to the mean of the clusters formed after the cluster assignment. And then it repeats the cycle. The centroids can no longer move around after a certain number of steps and then the iterations can end.

Implementation

Step 1: Importing of Libraries

```
from sklearn import datasets
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.cluster import KMeans
```

Step 2: Loading the data

```
iris = datasets.load_iris ()
```

Step 3: Defining the target and Predictors

```
X = iris.data[:, :2]
```

```
y = iris.target
```

Step 4: Visualizing the data

```
plt.scatter (X[:,0], X[:,1], c=y, cmap='gist_rainbow')
plt.xlabel ('Sepal Length', fontsize=18)
plt.ylabel ('Sepal Width', fontsize=18)
```

Output: as in Fig. 9.13

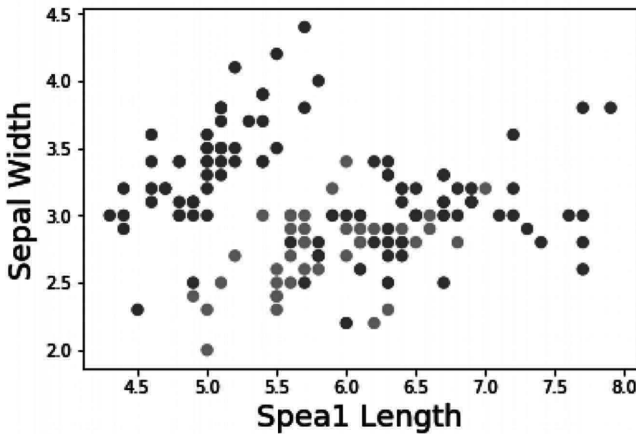


Fig. 9.13 Program outcome

Step 5: Loading and Compiling the model

```
km = KMeans (n_clusters = 3, n_jobs = 4, random_state=21)
km.fit (X)
```

Output:

```
KMeans (algorithm='auto', copy_x=True, init='k-means++',
max_iter=300, n_clusters=3, n_init=10, n_jobs=4,
precompute_distances='auto', random_state=21, tol=0.0001,
verbose=0)
```

Step 6: Getting Centroid

```
centers = km.cluster_centers_
print (centers)
```

Output:

```
[[5.77358491 2.69245283]
 [5.006 3.428 ]
 [6.81276596 3.07446809]]
```

Step 7: Comparing the original data and clustered data

```
#this will tell us to which cluster does the data observations
belong.
new_labels = km.labels_
# Plot the identified clusters and compare with the answers
fig, axes = plt.subplots (1, 2, figsize= (16,8))
axes[0].scatter (X[:, 0], X[:, 1], c=y, cmap='gist_rainbow',
edgecolor='k', s=150)
axes[1].scatter (X[:, 0], X[:, 1], c=new_labels, cmap='jet',
```

```

edgecolor='k', s=150)
axes[0].set_xlabel ('Sepal length', fontsize=18)
axes[0].set_ylabel ('Sepal width', fontsize=18)
axes[1].set_xlabel ('Sepal length', fontsize=18)
axes[1].set_ylabel ('Sepal width', fontsize=18)
axes[0].tick_params (direction='in', length=10, width=5,
                    colors='k', labels=20)
axes[1].tick_params (direction='in', length=10, width=5,
                    colors='k', labels=20)
axes[0].set_title ('Actual', fontsize=18)
axes[1].set_title ('Predicted', fontsize=18)

```

Output: as in Fig. 9.14

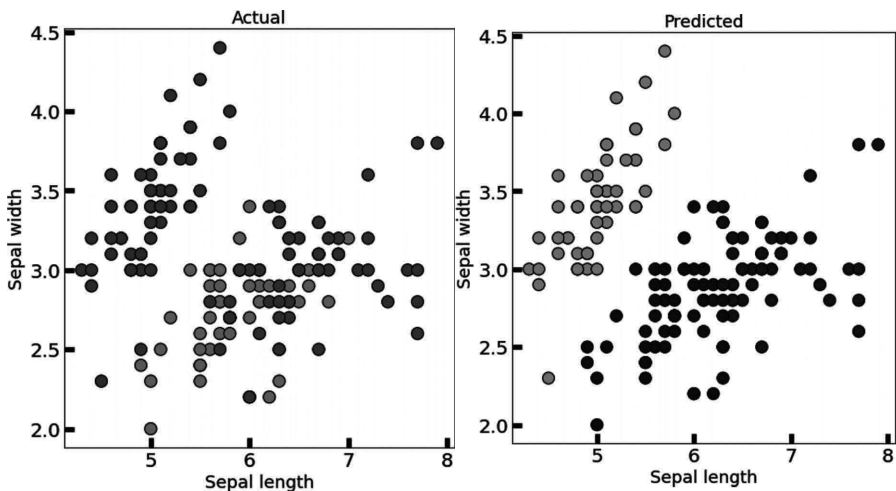


Fig. 9.14 Program outcome

Pros:

- K means are straightforward and machine efficient.
- It is highly intuitive and quick to imagine the effects.

Cons:

- K-means are strongly reliant upon scale and are not designed for data of different densities and shapes.
- More subjective is the estimation of performance. It needs much more human evaluation than metrics of confidence.

Random Forest

This is a supervised algorithm of classification. The random forest algorithm has the advantage that it could be used for problems of classification as well as regression. This is essentially the set of decision-making trees (forests) or the decision-making trees ensemble. The basic principle of the random forest is that each trees are graded and that the forest selects the best grades.

```
from sklearn.ensemble import RandomForestClassifier
#Import scikit-learn dataset library
from sklearn.model_selection import train_test_split
from sklearn import datasets
import pandas as pd
#Import scikit-learn metrics module for accuracy calculation
from sklearn import metrics
#Import scikit-learn metrics module for accuracy calculation
from sklearn import metrics
```

```
iris = datasets.load_iris ()  
  
# print the label species (setosa, versicolor, virginica)  
print (iris.target_names)  
  
# print the names of the four features  
print (iris.feature_names)
```

```
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
'petal width (cm)']

# print the iris data (top 5 records)
print (iris.data[0:5])

# print the iris labels (0:setosa, 1:versicolor, 2:virginica)
print (iris.target)
```

[illegible]


```
clf.fit (X_train,y_train)
```

```
y_pred=clf.predict (X_test)
```

Step 5: Seeing model accuracy and prediction

```
# Model Accuracy, how often is the classifier correct?
```

```
print ("Accuracy:",metrics.accuracy_score (y_test, y_pred))
```

Output:

Accuracy: 1.0

```
species_idx = clf.predict ([[3, 5, 4, 2]])[0]
```

```
iris.target_names[species_idx]
```

Output:

'versicolor'

Pros:

- For classification and regression functions, the random forest classifier may be used.
- The missing values can be managed.
- Even if we have more trees in the forest, this will not over suit the pattern.

Cons:

- Inherently, an ensemble model is less interpretable than a decision tree.
- A large number of deep trees can cost a great deal (but can be paralleled) and use a great deal of memory.
- Predictions are slower, which can pose implementation problems.

Summary

Machine learning algorithms are being used for so many purposes and which help us to get useful result by analyzing the data available from several resources. Here we have the different algorithms of machine learning like linear regression, logistic regression, decision tree etc., which have discussed in this chapter with implementation using real time dataset name Irish Dataset which is most famous dataset which consist of data of flowers. From the implementation part of the algorithm we can understand how these algorithms can be implemented in real time. pros and cons of algorithms also have been stated for better understanding.